JAVA PROBLEM SHEET - 7

COLLECTIONS API

QUESTION: 1
Write a program to store employee data using collection object called ArrayList. Also, perform the operations like Find the employees age over 50, Print the employees from USA and Sort the employees by country.
CODE:

```
import java.util.*;
class Employee {
String name, country;
int age;
Employee(String name, int age, String country) {
this.name = name;
this.age = age;
this.country = country;
}
@Override
public String toString() {
return name + " | " + age + " | " + country;
}
}
public class EmployeeArrayList {
public static void main(String[] args) {
ArrayList<Employee> employees = new ArrayList<>();
employees.add(new Employee("John", 55, "USA"));
employees.add(new Employee("Amit", 45, "India"));
employees.add(new Employee("Maria", 52, "USA"));
employees.add(new Employee("Chen", 34, "China"));
employees.add(new Employee("Lara", 29, "UK"));
System.out.println("\nEmployees age > 50:");

for (Employee e : employees) {
if (e.age > 50) System.out.println(e);
}
System.out.println("\nEmployees from USA:");
for (Employee e : employees) {
if (e.country.equalsIgnoreCase("USA")) System.out.println(e);
}
System.out.println("\nEmployees sorted by country:");
employees.sort(Comparator.comparing(emp -> emp.country));
for (Employee e : employees) {
```

```
System.out.println(e);
}
}
}
```

Op:

```
Employees age > 50:
John | 55 | USA
Maria | 52 | USA

Employees from USA:
John | 55 | USA
Maria | 52 | USA

Employees sorted by country:
Chen | 34 | China
Amit | 45 | India
Lara | 29 | UK
John | 55 | USA
Maria | 52 | USA
```

2. Write a method minToFront(list) that takes an ArrayList of integers as a parameter and that moves the minimum value in the list to the front, otherwise preserving the order of the elements. For example, if available called list stores the following values: {3, 8, 92, 4, 2, 17, 9} and you make this call: minToFront(list); it should store the following values after the call: {2, 3, 8, 92, 4, 17, 9}. You may assume that the list stores at least one value.

```
import java.util.ArrayList;
public class MinToFront {
public static void minToFront(ArrayList<Integer> list) {
int minIndex = 0;
int minValue = list.get(0);
for (int i = 1; i < list.size(); i++) {
if (list.get(i) < minValue) {
minValue = list.get(i);
minIndex = i;
```

```
}
}
list.remove(minIndex);
list.add(0, minValue);
}
public static void main(String[] args) {
ArrayList<Integer> list = new ArrayList<>();
list.add(3);
list.add(8);
list.add(92);
list.add(4);
list.add(2);
list.add(17);
list.add(9);
System.out.println("Original list: " + list);
minToFront(list);
System.out.println("After minToFront: " + list);
}
}
```

OUTPUT:

```
Original list: [3, 8, 92, 4, 2, 17, 9]
After minToFront: [2, 3, 8, 92, 4, 17, 9]
```

3. Write a program to find the duplicate elements stored in HashSet.

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
public class FindDuplicates {
public static void main(String[] args) {
// Sample list with duplicates (since HashSet can't store duplicates directly)
List<Integer> numbers = new ArrayList<>();
numbers.add(1);
numbers.add(2);
numbers.add(3);
numbers.add(2);
numbers.add(4);
numbers.add(3);
numbers.add(5);
Set<Integer> uniqueSet = new HashSet<>();
```

```
Set<Integer> duplicates = new HashSet<>();
for (Integer num : numbers) {
if (!uniqueSet.add(num)) {
duplicates.add(num);
}
}
System.out.println("Duplicate elements: " + duplicates);
}
}
```

Output:

```
Duplicate elements: [2, 3]
```

4)

4. Write a program to find the numbers greater than 50 and smallest and largest number in the array. Use TreeSet collection object.

```
import java.util.*;

public class Main {
public static void main(String[] args) {
int[] arr = {12, 67, 45, 89, 23, 51, 100, 34};

TreeSet<Integer> set = new TreeSet<>();
for (int num : arr) {
set.add(num);
}

System.out.println("Numbers greater than 50: " + set.tailSet(51));
System.out.println("Smallest number: " + set.first());
System.out.println("Largest number: " + set.last());
}
}
```

```
Numbers greater than 50: [51, 67, 89, 100]
Smallest number: 12
Largest number: 100
```

5.Design a PhoneBook using a HashMap to store contact information, where the key is the contact name, and the value is the phone number. The program presents a menu with the

following options:
Add Contact: Allows you to add a new contact to the phone book.
Lookup Contact: Lets you search for a contact by name and displays their phone number.
List: Display all the contacts from the phone Book
Edit: Allows to edit the phone number of the given name.
Sort: Display sorted list of all the contacts from the phone Book
Code:

```
import java.util.*;
public class PhoneBook {
static HashMap<String, String> contacts = new HashMap<>();
static Scanner sc = new Scanner(System.in);
public static void main(String[] args) {
while (true) {
System.out.println("\nPhoneBook Menu:");
System.out.println("1. Add Contact");
System.out.println("2. Lookup Contact");
System.out.println("3. List Contacts");
System.out.println("4. Edit Contact");
System.out.println("5. Sort Contacts");
System.out.println("6. Exit");
System.out.print("Choose: ");
int choice = sc.nextInt();
sc.nextLine();
switch (choice) {
case 1 -> addContact();
case 2 -> lookupContact();
case 3 -> listContacts();
case 4 -> editContact();
case 5 -> sortContacts();
case 6 -> System.exit(0);
default -> System.out.println("Invalid choice.");
}

}
}
static void addContact() {
System.out.print("Enter name: ");
String name = sc.nextLine();
System.out.print("Enter phone: ");
String phone = sc.nextLine();
contacts.put(name, phone);
}
static void lookupContact() {
System.out.print("Enter name to search: ");
```

```
String name = sc.nextLine();
System.out.println("Phone: " + contacts.getOrDefault(name, "Not
found"));
}
static void listContacts() {
for (Map.Entry<String, String> entry : contacts.entrySet()) {
System.out.println(entry.getKey() + ": " + entry.getValue());
}
}
static void editContact() {
System.out.print("Enter name to edit: ");
String name = sc.nextLine();
if (contacts.containsKey(name)) {
System.out.print("Enter new phone: ");
String phone = sc.nextLine();
contacts.put(name, phone);
} else {
System.out.println("Contact not found.");
}
}
static void sortContacts() {
TreeMap<String, String> sorted = new TreeMap<>(contacts);
for (Map.Entry<String, String> entry : sorted.entrySet()) {
System.out.println(entry.getKey() + ": " + entry.getValue());
}
}
}
```

Output:

```
PhoneBook Menu:
1. Add Contact
2. Lookup Contact
3. List Contacts
4. Edit Contact
5. Sort Contacts
6. Exit
Choose: 3
abcdsdsd: 34567856789
```

6. Logistics and courier companies (like FedEx, BlueDart, DTDC) need to manage

packages, delivery agents, and tracking information efficiently. The system should allow adding new packages, assigning delivery agents, updating delivery status, and tracking packages until they reach the customer. Each package has a unique tracking ID, sender info, receiver info, source, destination, and status (Booked, In Transit, Delivered). Maintain a list of delivery agents to assign packages and track the same. Also maintain delivery priority based on urgency. Generate the reports based on the queries like list all packages for a given customer, display packages based on status, display delivered packages sorted by date and finally display agent wise delivery report.

Source code:

```
import java.util.*;
class Package {
String trackingID, sender, receiver, source, destination, status, priority;
Date date;
Package(String trackingID, String sender, String receiver, String source, String destination,
String priority) {

this.trackingID = trackingID;
this.sender = sender;
this.receiver = receiver;
this.source = source;
this.destination = destination;
this.priority = priority;
this.status = "Booked";
this.date = new Date();
}
public String toString() {
return trackingID + " | " + sender + " -> " + receiver + " | " + source + " -> " + destination +
" | " + status + " | " + priority;
}
}
public class CourierSystem {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
ArrayList<Package> packages = new ArrayList<>();
HashMap<String, String> agentAssign = new HashMap<>();
packages.add(new Package("PKG1", "Alice", "Bob", "Delhi", "USA", "High"));
packages.add(new Package("PKG2", "Charlie", "David", "Mumbai", "UK", "Normal"));
agentAssign.put("PKG1", "Agent1");
agentAssign.put("PKG2", "Agent2");
System.out.println("All Packages:");
for (Package p : packages) System.out.println(p);
System.out.println("\nDelivered Packages Sorted by Date:");
packages.get(0).status = "Delivered";
packages.sort(Comparator.comparing(p -> p.date));
```

```
for (Package p : packages) {
if (p.status.equals("Delivered")) System.out.println(p);
}
System.out.println("\nAgent-wise report:");
for (String id : agentAssign.keySet()) {
System.out.println(agentAssign.get(id) + " -> " + id);
}
}
}
```

```
All Packages:
PKG1 | Alice -> Bob | Delhi -> USA | Booked | High
PKG2 | Charlie -> David | Mumbai -> UK | Booked | Normal

Delivered Packages Sorted by Date:
PKG1 | Alice -> Bob | Delhi -> USA | Delivered | High

Agent-wise report:
Agent1 -> PKG1
Agent2 -> PKG2
```